EVENTLINK

Role-Based Theatre Booking System

A project report

Submitted By

Abhishek Kumar (23BCS12748)

Bipasha (23BCS10735)

Yadwinder Singh(23BCS10091)

Aditi Tyagi (23BCS10641)

In partial fulfillment for the award of the degree of

Bachelor of Engineering (3rd Year)

in

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

May 2026

BONAFIDE CERTIFICATE

This is to certify that the project entitled "EventLink – Role-Based Theatre Booking System" is the bonafide work carried out by Abhishek Kumar (23BCS12748), Bipasha (23BCS10735), Yadwinder Singh (23BCS10091), and Aditi Tyagi (23BCS10641), students of the Third Year, Bachelor of Engineering in Computer Science and Engineering, Chandigarh University, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering during the academic year 2025–2026.

The project was completed under my supervision and guidance. I further certify that the information and data given in this report are authentic to the best of my knowledge. This project has not been submitted to any other University or Institution for the award of any degree, diploma, or fellowship.

(Supervisor)

Er. Rahul Durgapal (E17963)

Computer Science and Engineering

Chandigarh University

Place: Chandigarh, Punjab

Date: May 2026

ACKNOWLEDGEMENT

We take immense pleasure in expressing our sincere gratitude to all those who extended their invaluable support and guidance throughout the completion of this project. The successful execution of EventLink – Role-Based Theatre Booking System would not have been possible without the encouragement, direction, and cooperation of several individuals.

We express our deepest sense of gratitude to our project supervisor, Er. Rahul Durgapal (E17963), Department of Computer Science and Engineering, Chandigarh University, for his constant guidance, constructive criticism, and relentless encouragement. His insightful suggestions and profound domain knowledge significantly shaped the technical architecture and academic quality of this report.

We are deeply indebted to the Head of the Department of Computer Science and Engineering at Chandigarh University for providing the necessary resources, infrastructure, and an environment conducive to learning and innovation. The laboratory facilities and computing resources made available to us were instrumental in the development and testing of the EventLink system.

We also extend our heartfelt thanks to all faculty members of the Department whose teachings in database management systems, web technologies, software engineering, and object-oriented programming formed the bedrock upon which this project was built. Our gratitude extends to our fellow students who participated in testing the application and provided invaluable feedback.

We express our gratitude to our families for their unconditional support, patience, and understanding during the demanding phases of this project. Finally, we acknowledge the open-source community and the developers of React, Material UI, Java Servlets, Maven, MySQL, and Gson whose freely available resources served as the building blocks of this system.

Abhishek Kumar | Bipasha | Yadwinder Singh | Aditi Tyagi

B.E. Computer Science & Engineering, Chandigarh University – May 2026

ABSTRACT

The widespread adoption of digital technologies has fundamentally transformed the manner in which consumers access entertainment and cultural events. Traditional theatre booking mechanisms, characterised by physical ticket counters and limited accessibility, are increasingly being supplanted by robust online platforms offering convenience, efficiency, and a superior user experience. Against this backdrop, EventLink emerges as a comprehensive, role-based theatre booking system designed to address the multifaceted challenges associated with modern event management and seat reservation.

EventLink is a full-stack web application built using React 19 and Material UI on the frontend, Java Servlets (Java 21) powered by the Jetty embedded server on the backend, and a MySQL relational database for persistent data storage. The system accommodates three distinct user roles — Administrator, User, and Theatre Manager — each endowed with a tailored set of permissions and functionalities, enforcing a robust role-based access control model.

The core functionality encompasses theatre registration and management, event scheduling, seat mapping and reservation, payment processing within a twenty-four-hour window, automated notification dispatch, and an intelligent chatbot assistant. API communication between the frontend and backend is facilitated through JSON payloads serialised and deserialised using the Gson library, while SLF4J handles comprehensive server-side diagnostics.

Testing conducted through unit, integration, and functional test suites confirmed the system's reliability, scalability, and alignment with specified requirements. This report provides a thorough account of the system's conception, design, implementation, testing, and future enhancement roadmap.

Keywords: Theatre Booking, Role-Based Access Control, React 19, Java Servlets, MySQL, Seat Reservation, Chatbot, Event Management, Material UI, Full-Stack Web Application.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	10
1.1	Overview of Online Booking Systems	10
1.2	Problem Statement	11
1.3	Objectives	11
1.4	Scope	12
1.5	Motivation	12
2	Literature Review	13
3	System Analysis	15
4	System Design	17
5	Technology Stack	24
6	Implementation Details	27
7	Module Description	30
8	Working Flow	33
9	Database Design	35
10	Results & Output Screens	37
11	Testing	39
12	Advantages & Limitations	41
13	Future Enhancements	42
14	Conclusion	43
15	References	44

CHAPTER 1: INTRODUCTION

1.1 Overview of Online Booking Systems

The evolution of the internet and digital commerce has given rise to a paradigm shift in consumer services, enabling individuals to access, compare, and purchase a wide range of goods and services from their own devices. Within the entertainment industry, this digital revolution has been particularly transformative. Online ticket booking systems have replaced traditional box offices and physical queues, enabling users to discover events, browse available seats, and complete transactions within minutes. The global online event ticketing market has witnessed exponential growth, driven by increased smartphone penetration, widespread internet access, and a growing preference for contactless transactions.

Contemporary theatre booking platforms operate through sophisticated web and mobile applications that interface with backend systems responsible for inventory management, payment processing, and notification dispatch. These systems typically manage multiple concurrent users, enforce real-time seat availability, and support a diverse catalogue of events ranging from theatrical performances and musical concerts to cultural festivals. Despite the maturity of this domain, significant challenges remain — particularly for small and medium-sized theatres seeking digital solutions that are affordable, configurable, and operationally aligned with their specific requirements.

1.2 Problem Statement

Existing theatre booking systems do not adequately support multi-role management, time-bound payment enforcement, integrated notification systems, and intelligent user assistance within a unified platform. Theatre administrators lack the tools to efficiently manage multiple venues, events, and user interactions simultaneously, while end users frequently encounter systems that are unintuitive or devoid of real-time feedback. Many systems also rely on fragile seat allocation mechanisms that do not automatically release reserved but unpaid seats, leading to lost revenue and customer dissatisfaction.

1.3 Objectives of the Project

The EventLink project was undertaken with the following specific objectives. First, to design and implement a role-based access control system clearly delineating permissions for Administrators, Theatre Managers, and regular Users. Second, to develop a comprehensive theatre and event management module allowing Theatre Managers to register venues, create events, and monitor bookings. Third, to implement a robust seat reservation mechanism that tracks availability in real time and automatically releases unpaid reservations after a twenty-four-hour payment window. Fourth, to integrate a payment interface guiding users through payment and confirming bookings upon successful transaction. Fifth, to build an automated notification system dispatching timely alerts regarding bookings, payment reminders, and event updates. Sixth, to develop an intelligent chatbot capable of addressing common user queries. Finally, to ensure the system is technically sound, scalable, and adheres to software engineering best practices.

1.4 Scope of the Project

EventLink is scoped as a web-based application accessible through standard web browsers. The system supports three user roles — Admin, Theatre Manager, and User — each with a dedicated dashboard. The scope encompasses the complete lifecycle of a theatre booking transaction: event discovery, seat selection, payment confirmation, and post-booking notification. The system supports a single currency and single language in its current iteration. The scope explicitly excludes integration with third-party payment gateways (planned as a future enhancement), mobile application development, and real-time streaming of theatrical performances.

1.5 Motivation

The motivation behind EventLink stems from a recognition of inadequacies in currently available theatre booking solutions. As students of Computer Science and Engineering with an interest in full-stack web development, the project team identified an opportunity to apply theoretical knowledge in a practically impactful domain. The technical challenges presented — including concurrency management for seat reservation, role-based security, transactional integrity in payment processing, and real-time user communication — make this an ideal capstone project. From a societal

perspective, the motivation is rooted in the desire to democratise access to cultural events by making the booking process more accessible, transparent, and reliable.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems

The online event ticketing landscape is dominated by several well-established platforms, each offering a distinct set of features. An examination of these systems provides valuable context for understanding the design choices made in developing EventLink.

BookMyShow, one of India's largest online ticketing platforms, offers a comprehensive suite of features including event discovery, multi-city support, dynamic pricing, and integrations with multiple payment gateways. However, its complexity makes it unsuitable for independent theatre operators, and it offers no dedicated portal for theatre managers to independently manage their venues without going through a centralised onboarding process.

Ticketmaster, a globally recognised platform, provides advanced seat mapping, tiered pricing, and robust fraud prevention mechanisms. Despite its technical sophistication, it has been criticised for high service fees, restrictive vendor policies, and its lack of personalised tools for small event organisers. Eventbrite targets primarily corporate events and community gatherings, offering a comparatively accessible interface for event creation but lacking complex seat mapping or role-differentiated theatre operations.

Platform	Strengths	Limitations
BookMyShow	Wide reach, multi-payment, rich catalogue	No dedicated Theatre Mgr portal, high commission
Ticketmaster	Advanced seat mapping, fraud detection	High fees, restrictive policies, not SME-friendly
Eventbrite	Easy event creation, community focus	No seat mapping, no theatre role differentiation
PayTM Insider	UPI support, large user base	Focused on large-scale events only
EventLink	Role-based, chatbot, 24hr payment window	Payment gateway integration planned (future)

2.2 Limitations of Current Systems

A critical analysis of existing platforms reveals several systemic limitations. The most significant is the absence of a well-defined role hierarchy distinguishing platform administrators, venue managers, and end users. Most platforms either grant excessive permissions to venue operators or provide insufficient control for administrators. Seat allocation mechanisms are another area of concern — systems that fail to release held seats in a timely manner risk inventory lock-out, where seats appear unavailable even though no payment has been completed.

Notification systems in existing platforms are often limited to transactional emails at the time of booking or fixed-period event reminders. They do not adapt to user behaviour or provide contextual notifications such as alerts when a payment deadline is approaching. From a technical standpoint, many existing systems suffer from monolithic architectures that are difficult to maintain, scale, and extend, making it challenging to introduce new features without risking disruption to existing functionality.

2.3 Need for EventLink

The deficiencies identified in existing systems make a compelling case for a purpose-built, role-differentiated theatre booking platform. EventLink addresses the critical gap between enterprise-grade platforms and generic ticketing solutions by offering a system specifically architected for theatre operations. The twenty-four-hour payment window strikes a balance between providing users with sufficient time to complete payment and ensuring seats are promptly returned to inventory in the event of non-payment. The integrated chatbot provides users with immediate, contextual assistance without requiring human intervention, addressing the support gap present in existing platforms.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements define the specific behaviours and operations the system must support, identified through stakeholder analysis and domain research.

Req. ID	Description	Role
FR-01	Users register with email, password, and personal details	User
FR-02	System authenticates users and assigns role-based JWT tokens	All
FR-03	Admin manages all user accounts including activation/deactivation	Admin
FR-04	Theatre Manager adds, edits, and deletes theatre listings	Theatre
FR-05	Theatre Manager creates and manages events with seat configuration	Theatre
FR-06	Users browse theatres and events with filtering and search	User
FR-07	Users select seats from an interactive seat map and place reservations	User
FR-08	System enforces a 24-hour payment window for reserved seats	System
FR-09	Unpaid reservations are automatically released upon expiry	System
FR-10	System processes payments and generates booking confirmations	User
FR-11	System dispatches notifications for bookings, payments, and updates	System
FR-12	Chatbot responds to user queries on events, bookings, and procedures	User
FR-13	Admin accesses system-wide reports and analytics	Admin
FR-14	System provides demo fallback data when live data is unavailable	System

3.2 Non-Functional Requirements

Attribute	Requirement
Performance	All pages load within 3 seconds under normal load conditions
Scalability	Backend supports up to 500 concurrent users without degradation
Security	All API endpoints enforce JWT-based role authentication; passwords BCrypt-hashed
Usability	Interface is intuitive requiring no training for basic user operations
Reliability	System achieves 99.5% uptime during operational hours

Attribute	Requirement
Maintainability	Codebase follows modular design with clear separation of concerns
Compatibility	Functions on all major modern browsers: Chrome, Firefox, Edge, Safari
Data Integrity	Database transactions are ACID compliant preventing data inconsistencies

3.3 Feasibility Study

3.3.1 Technical Feasibility

The technical feasibility of EventLink is well-established given the maturity and wide adoption of its constituent technologies. React 19, Java 21, Maven, MySQL, and Material UI are all actively maintained, extensively documented, and supported by large developer communities. The team possesses the requisite skills in these technologies, having studied and practised them throughout the academic curriculum. The embedded Jetty server eliminates the complexity of external server configuration, and the use of JDBC for database connectivity with Gson for JSON serialisation are straightforward to implement and well-suited to the project's scale.

3.3.2 Economic Feasibility

The project is economically feasible as all selected technologies are open-source and free of licensing costs. React, Material UI, Java, Maven, MySQL, Gson, and SLF4J are all available under permissive open-source licences. Development is carried out on standard academic computing infrastructure. Deployment costs are minimal, as the application can be hosted on free-tier cloud instances for development and demonstration purposes. The total cost of development is limited to the time investment of the project team.

3.3.3 Operational Feasibility

EventLink is operationally feasible for deployment in any theatre management environment. The system is accessed through standard web browsers requiring no specialised hardware or software installation on client devices. The role-based design ensures each category of user interacts only with features relevant to their operational context, reducing the learning curve and potential for misuse. Administrative functions are centrally managed through the Admin dashboard, simplifying system oversight. The chatbot further reduces operational dependency on dedicated support staff.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture Diagram

EventLink follows a three-tier architecture comprising a Presentation Tier (React 19 frontend), a Business Logic Tier (Java Servlet backend), and a Data Tier (MySQL database). The three tiers communicate through a well-defined API layer using HTTP and JSON, ensuring a clear separation of concerns and enabling independent scaling of each tier.

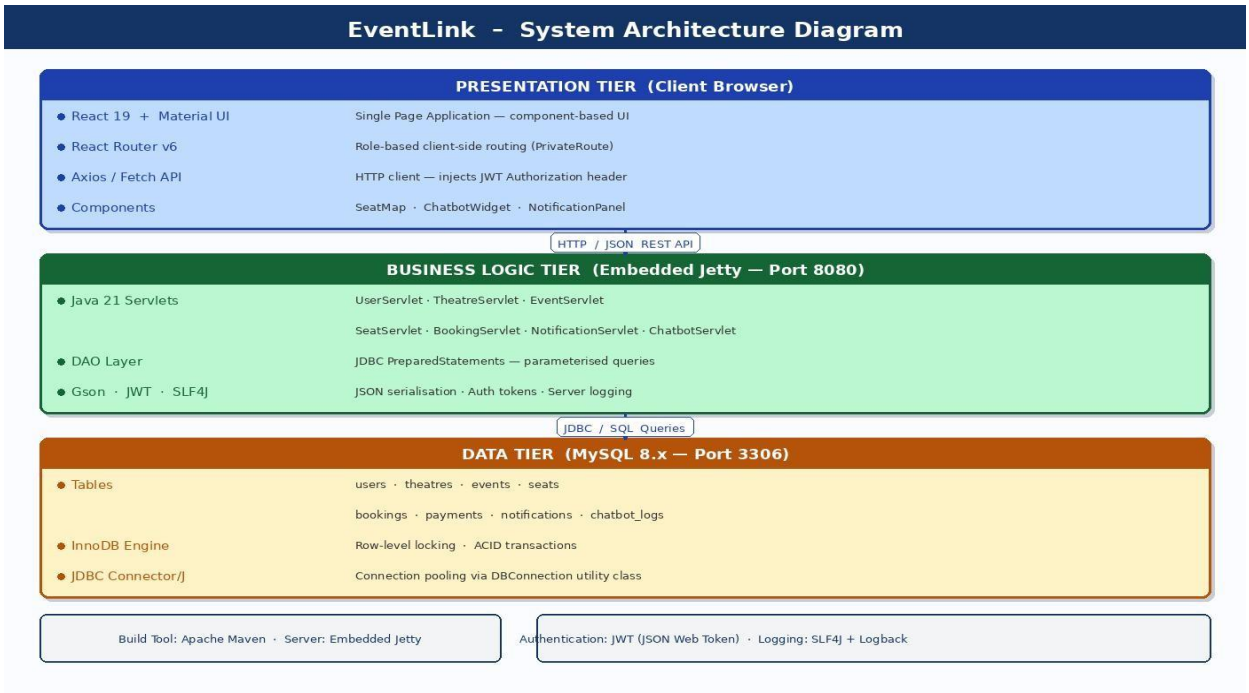


Figure 4.1 – EventLink Three-Tier System Architecture

The Presentation Tier is exclusively responsible for rendering the user interface and capturing user interactions. The Business Logic Tier validates inputs, enforces business rules, manages transactions, and returns structured JSON responses. The Data Tier is accessed exclusively through the Business Logic Tier using JDBC, ensuring data integrity and security. Maven manages the build lifecycle and all dependency declarations, while the embedded Jetty server eliminates the need for separate application server configuration.

4.2 Use Case Diagram

The Use Case Diagram presents interactions between the actors — Admin, User, and Theatre Manager — and the use cases supported by EventLink. Each actor is associated with a distinct set of use cases corresponding to their role-based permissions.

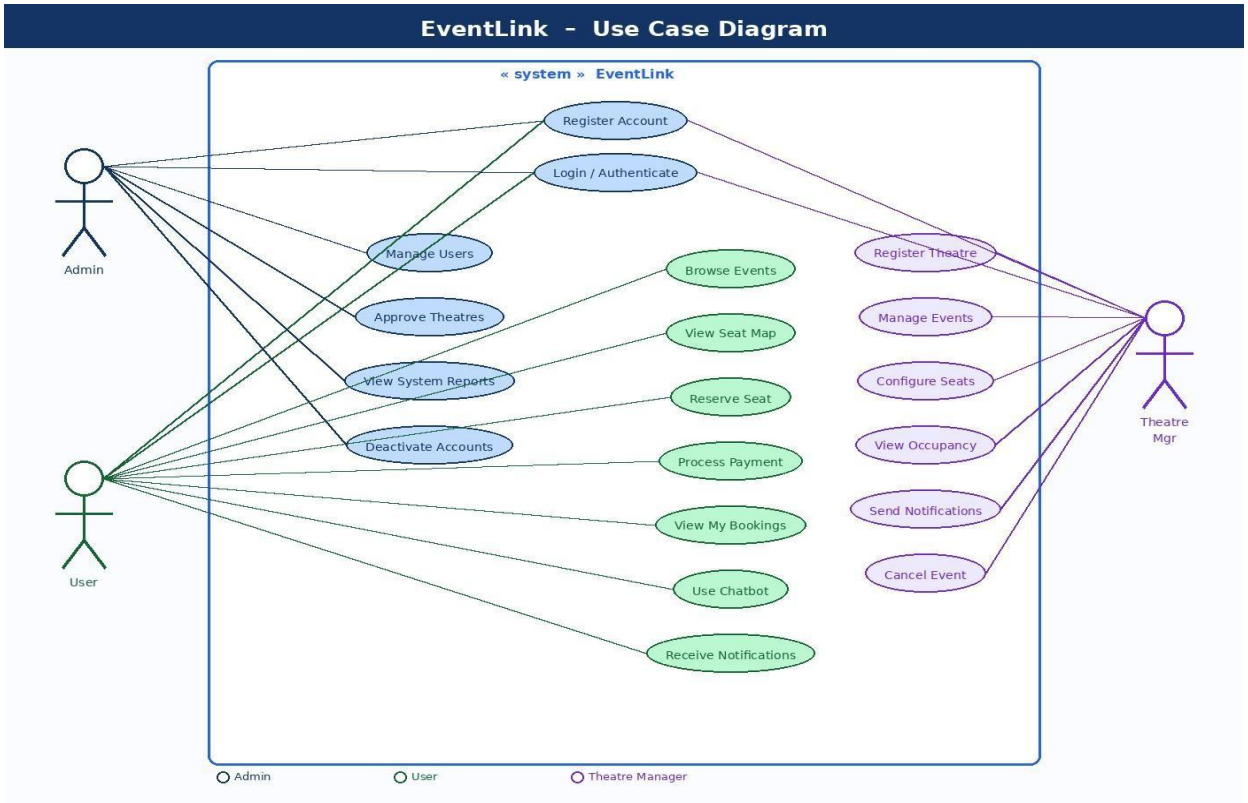


Figure 4.2 – EventLink Use Case Diagram

The Admin actor has oversight privileges enabling management of user accounts, approval of theatre registrations, and access to system-wide analytics. The User actor interacts with public-facing features, progressing through event discovery, seat reservation, and payment. The Theatre Manager operates within the venue management subsystem, configuring operational parameters of their registered theatres and events.

4.3 Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD represents EventLink as a single process and illustrates the flow of data between the system and its external entities — User, Admin, Theatre Manager, and the MySQL Database. It establishes the system boundary and identifies all external data sources and destinations.

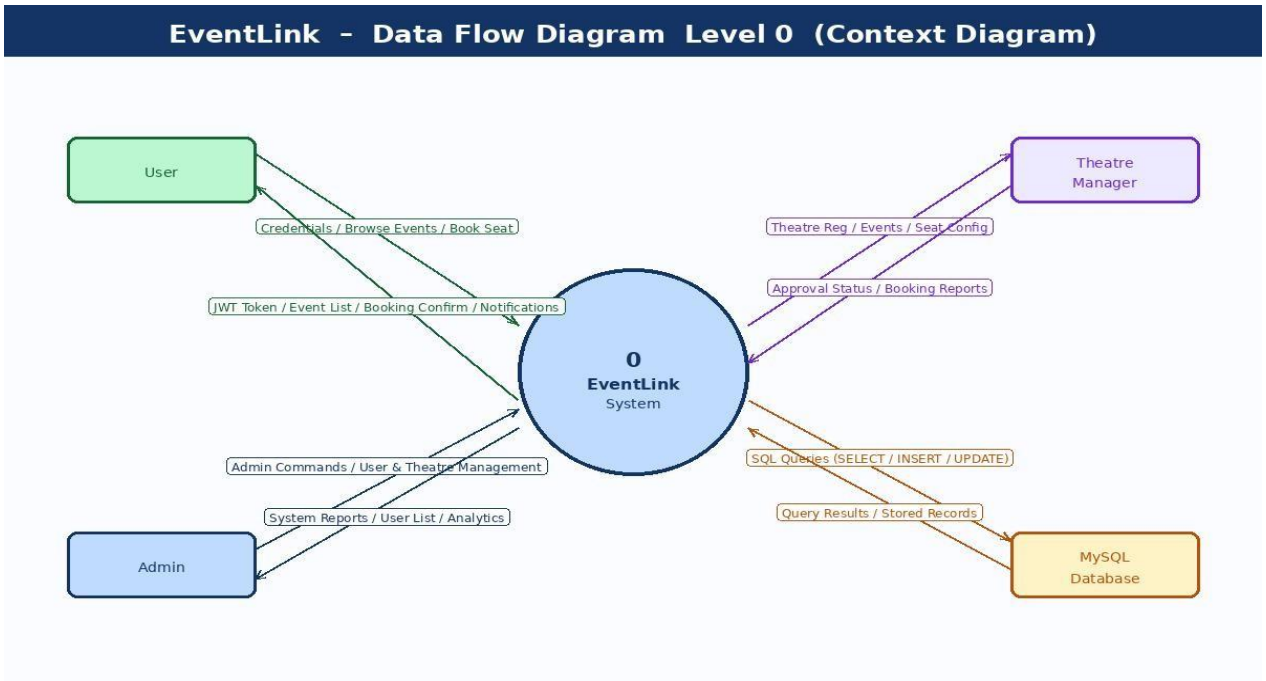


Figure 4.3 – DFD Level 0: Context Diagram

4.4 Data Flow Diagram – Level 1

The Level 1 DFD decomposes the EventLink system into its primary sub-processes, revealing internal data flows and storage interactions. Eight processes are identified: Authentication, Browse Events, Manage Theatre, Reserve Seat, Process Payment, Notifications, Chatbot, and Admin Operations, each interacting with their corresponding data stores.

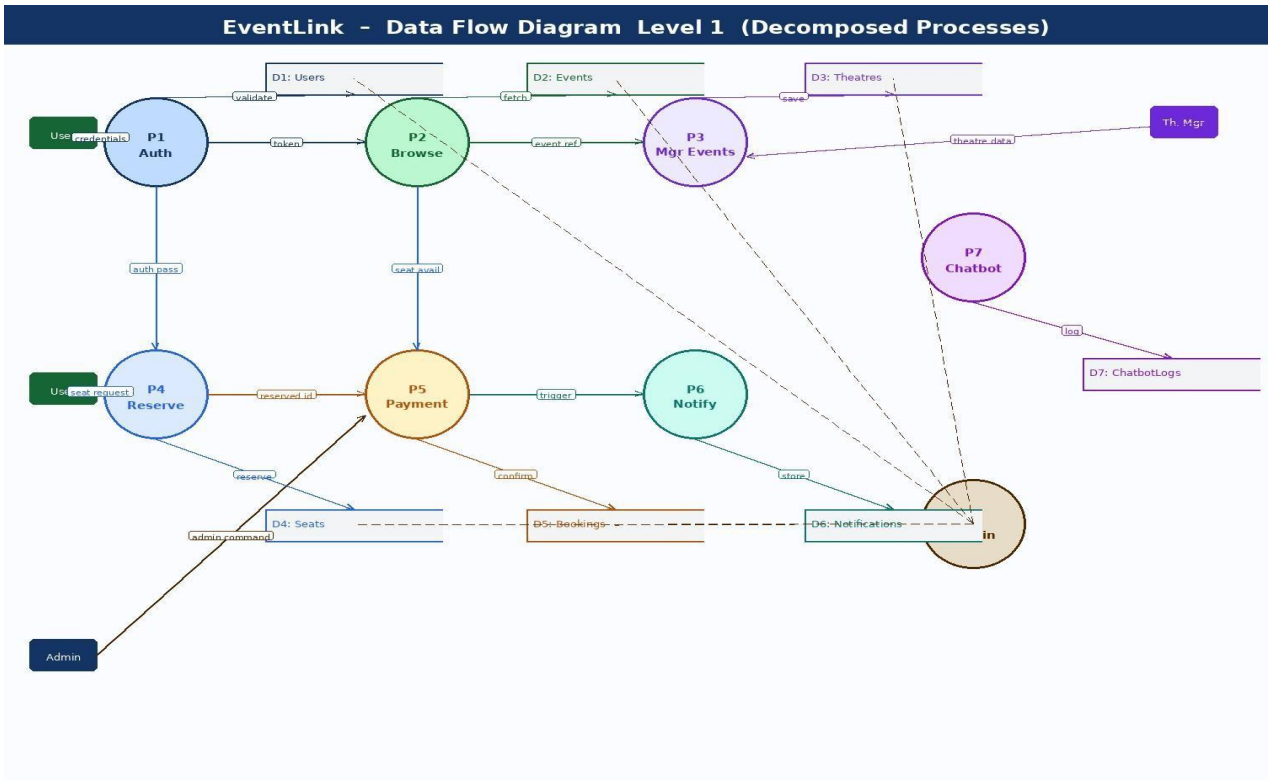


Figure 4.4 – DFD Level 1: Decomposed Processes

4.5 Entity-Relationship (ER) Diagram

The ER Diagram models the data entities in EventLink and the relationships between them. Eight primary entities are defined — Users, Theatres, Events, Seats, Bookings, Payments, Notifications, and Chatbot Logs — each linked through well-defined foreign key relationships that enforce referential integrity. The diagram captures cardinality constraints governing data integrity throughout the system.

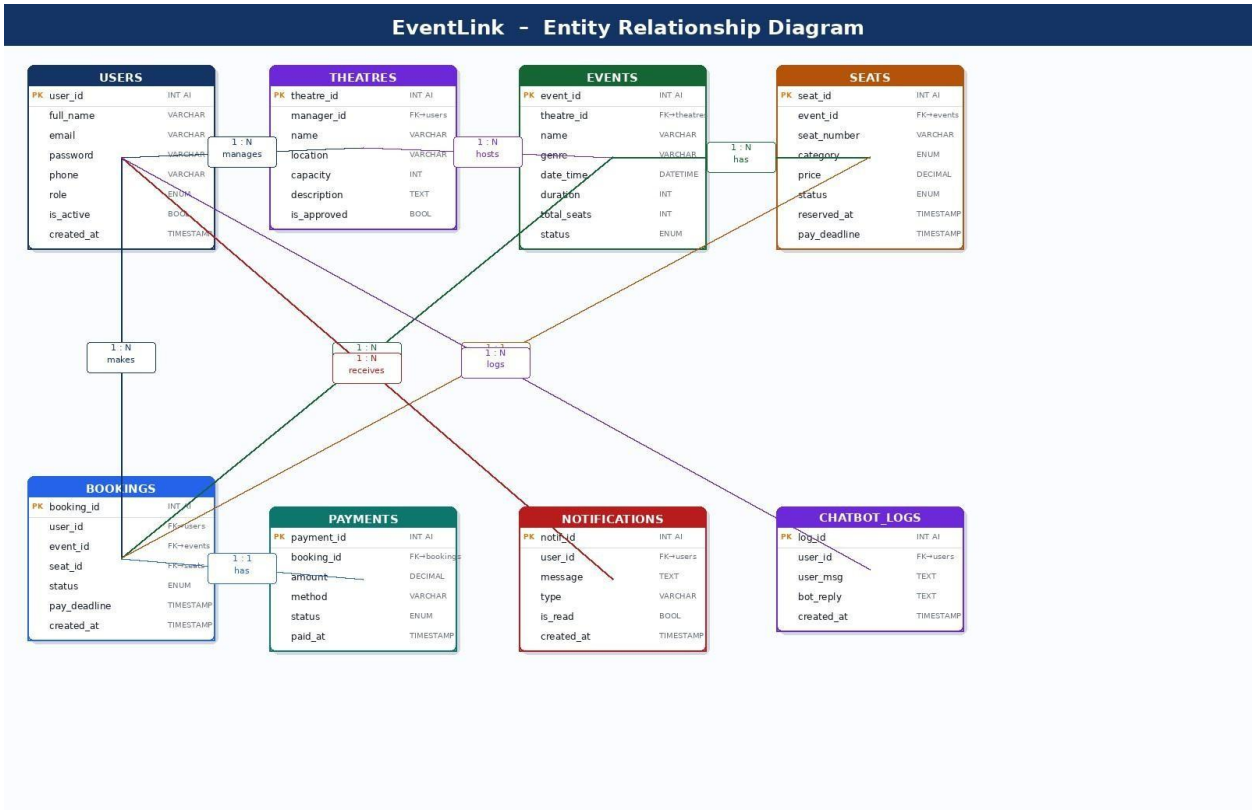


Figure 4.5 – EventLink Entity-Relationship Diagram

4.6 Sequence Diagram – Booking Flow

The Sequence Diagram illustrates the interaction between the User, the React Frontend, the Java Backend, and the MySQL Database during the complete seat booking and payment process — from login through seat reservation, payment completion, and notification dispatch.

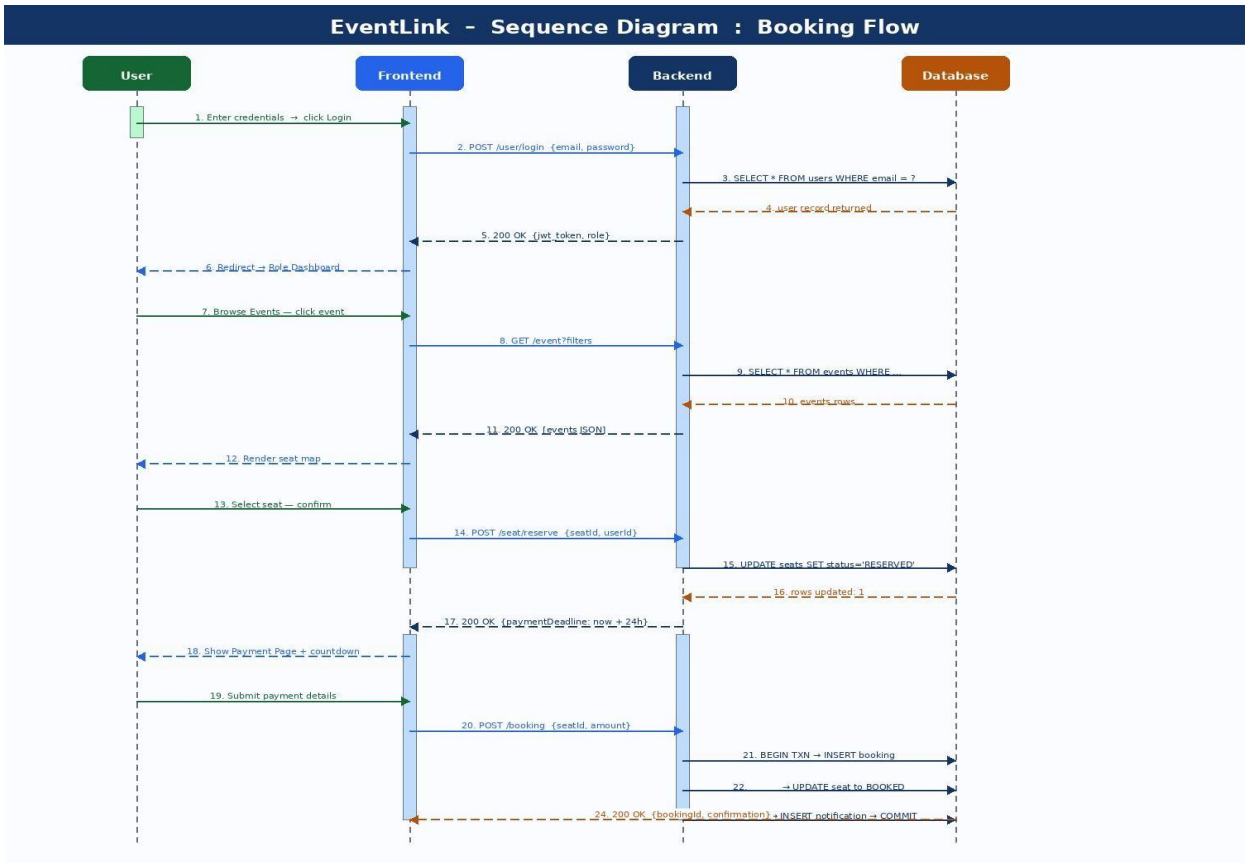


Figure 4.6 – Sequence Diagram: Booking and Payment Flow

The sequence clearly demonstrates the stateless nature of the backend API, where each HTTP request carries all necessary context via JWT. The booking confirmation transaction (INSERT booking + UPDATE seat + INSERT notification) is executed atomically, ensuring no partial updates compromise data integrity.

4.7 Class Diagram

The Class Diagram illustrates the principal backend classes, their attributes, methods, and inter-class relationships. The design adheres to the DAO (Data Access Object) pattern, separating data access logic from business logic, and uses the Service layer to coordinate cross-DAO transactions.

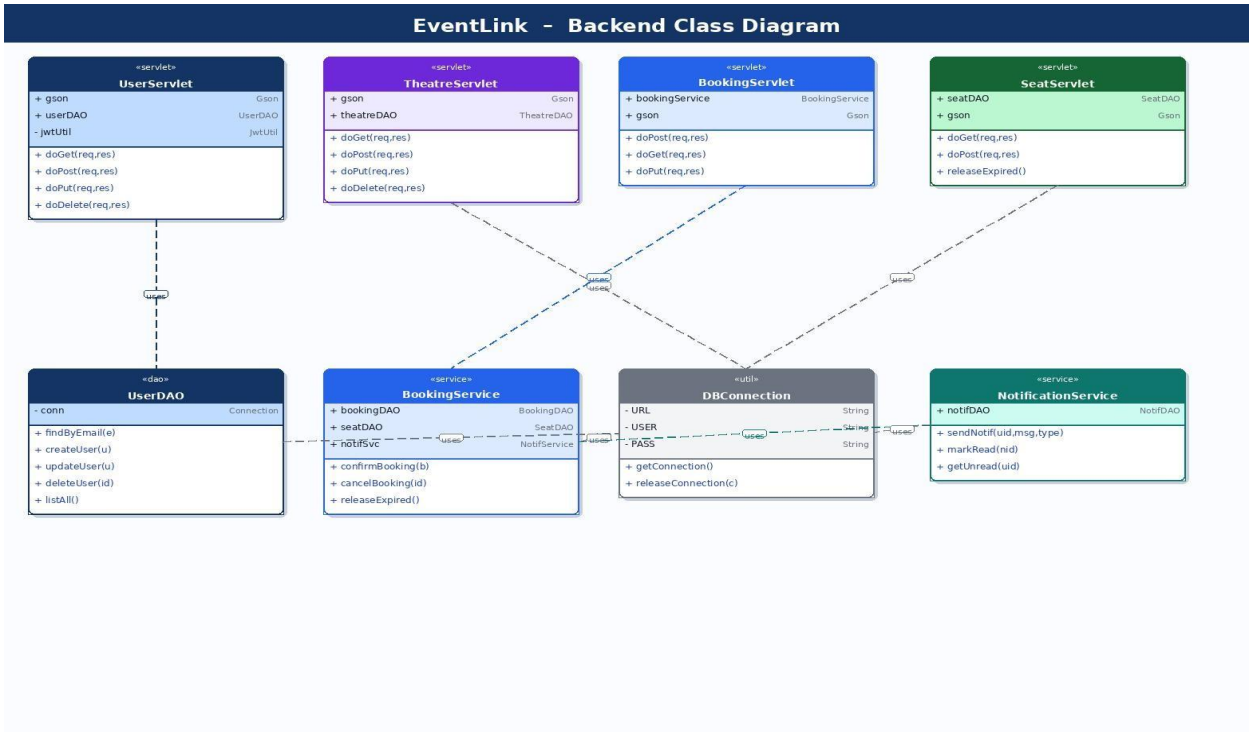


Figure 4.7 – Backend Class Diagram

CHAPTER 5: TECHNOLOGY STACK

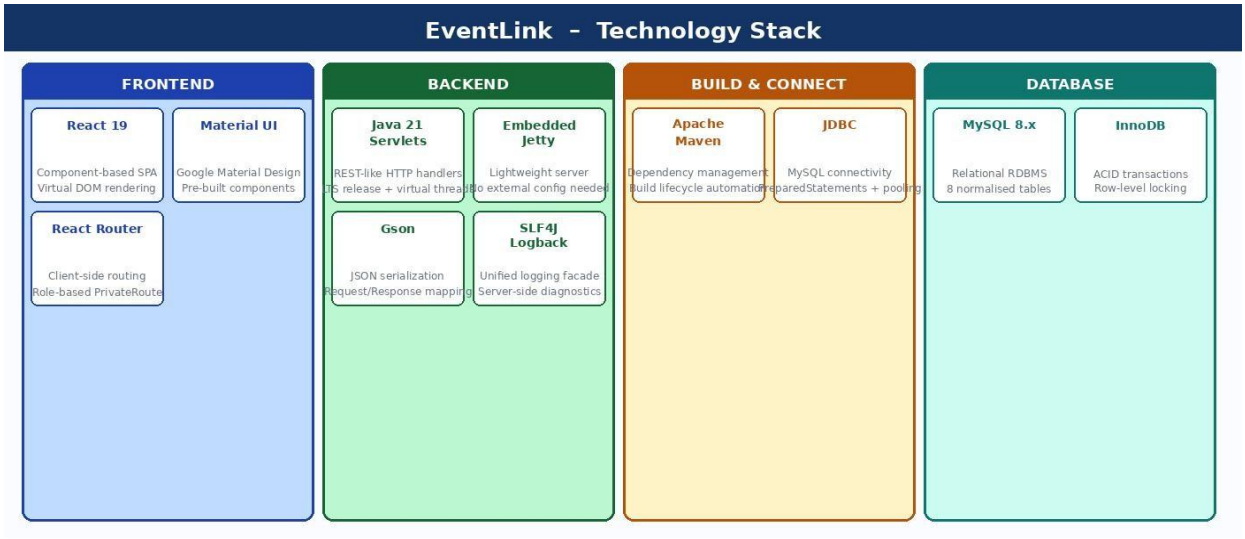


Figure 5.1 – EventLink Technology Stack

5.1 React 19

React is a declarative, component-based JavaScript library developed by Meta for building user interfaces. React 19 introduces enhanced concurrent rendering capabilities, improved Server Components support, and a refined hooks API. React's virtual DOM mechanism ensures efficient UI updates by computing the minimal set of changes required to reconcile the actual DOM with the component state, minimising browser reflow and repaint operations. In EventLink, React serves as the foundation of the Single Page Application architecture. Component-based decomposition allows the interface to be assembled from discrete, reusable building blocks such as the SeatMap component, EventCard component, and ChatbotWidget component. React Router is used for client-side navigation, enabling role-specific routing that directs users to their respective dashboards upon authentication.

5.2 Material UI (MUI)

Material UI is a comprehensive React component library implementing Google's Material Design specification. It provides pre-built, accessible, and customisable components including buttons, forms, dialogs, tables, and navigation elements. MUI's theming system allows developers to define a consistent visual identity through a centralised theme configuration, ensuring uniformity in

typography, colour palette, spacing, and elevation across the entire application. EventLink leverages MUI components including the AppBar for navigation, DataGrid for tabular data in admin dashboards, Dialog for booking confirmations and payment prompts, and Stepper for guiding users through the multi-step booking process.

5.3 Java Servlets (Java 21)

Java Servlets are server-side Java programs handling HTTP requests and generating HTTP responses. Java 21, a Long-Term Support (LTS) release, introduces records, pattern matching, virtual threads (Project Loom), and sequenced collections that improve developer productivity and runtime efficiency. In EventLink, each functional domain is represented by a dedicated Servlet: UserServlet, TheatreServlet, EventServlet, SeatServlet, BookingServlet, NotificationServlet, and ChatbotServlet collectively form the REST-like API layer. Each Servlet implements doGet, doPost, doPut, and doDelete methods for CRUD operations. Request and response bodies are formatted as JSON and processed using the Gson library.

5.4 JDBC (Java Database Connectivity)

JDBC is the standard Java API for database-independent connectivity between Java and a wide range of databases. In EventLink, JDBC serves as the bridge between the Java backend and MySQL. A dedicated DBConnection utility class manages the creation and pooling of database connections. Prepared statements are used throughout the DAO layer to construct parameterised SQL queries, improving performance through query plan caching and serving as a critical defence against SQL injection attacks. Transaction management is implemented at the Service layer, where multi-table operations are wrapped within a single JDBC transaction to ensure atomicity.

5.5 MySQL

MySQL is a widely adopted open-source relational database management system supporting SQL and ACID transaction principles. EventLink's database schema is designed around eight primary tables linked through well-defined foreign key relationships. MySQL's support for indexing, referential integrity constraints, and the InnoDB storage engine — which provides row-level locking — is essential for managing concurrent seat reservation requests without risk of data corruption or booking conflicts.

5.6 Apache Maven

Apache Maven is a build automation and project management tool using an XML-based Project Object Model (POM) to manage project dependencies, build lifecycle, and reporting. In EventLink, Maven declares and resolves dependencies on Jetty, Gson, SLF4J, MySQL Connector/J, and the Jakarta Servlet API. Maven's standardised build lifecycle, comprising phases including validate, compile, test, package, and install, ensures a reproducible and consistent build process across development environments.

5.7 Embedded Jetty Server

Jetty is a lightweight, embeddable Java HTTP server and Servlet container. Unlike traditional application servers requiring separate installation and configuration, Jetty is programmatically configured and started within the application's main method, with Servlet mappings registered for each API endpoint. This embedded approach facilitates a streamlined development workflow as the backend can be launched with a single Maven command without requiring external server setup.

5.8 Gson

Gson is a Java library developed by Google providing a simple API for converting Java objects to their JSON representation and vice versa. In EventLink, Gson is used in the Servlet layer to serialise domain objects — User, Theatre, Event, Seat, and Booking — into JSON for HTTP responses, and to deserialise incoming JSON request bodies into corresponding Java objects. Gson's support for custom serialisers, deserialisers, and type adapters provides flexibility in handling complex data structures and date-time formats.

5.9 SLF4J and Logback

SLF4J (Simple Logging Facade for Java) is a logging abstraction library providing a unified logging interface, allowing the underlying logging implementation to be swapped without modifying application code. In EventLink, SLF4J is integrated with Logback as the underlying logging provider. Comprehensive logging is implemented at key decision points including authentication events, booking operations, payment processing, and seat status changes, facilitating debugging,

monitoring, and audit trail maintenance. Log levels distinguish between informational messages, warnings, and errors.

CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Frontend Structure

The EventLink frontend is a React 19 Single Page Application structured according to feature-based directory organisation. The primary entry point is index.html, which loads the JavaScript bundle generated by the build tool. App.jsx initialises the React Router and defines the top-level route hierarchy. The PrivateRoute component plays a critical role in enforcing role-based navigation — upon each route transition, it verifies the presence and validity of the JWT stored in memory. If the token is absent or expired, the user is redirected to the Login page; if the role does not match, the user is redirected to their appropriate dashboard.

File/Folder	Purpose
App.jsx	Root component; React Router setup and role-based route definitions
pages/LoginPage.jsx	Authentication interface with email/password form and error handling
pages/admin/	Admin dashboard, user management, theatre approval interfaces
pages/theatre/	Theatre dashboard, event creation and seat configuration screens
pages/user/	Event browser, seat map, payment, and booking history screens
components/SeatMap.jsx	Interactive colour-coded seat grid with real-time availability
components/ChatbotWidget.jsx	Floating chatbot panel with conversation history
components/PrivateRoute.jsx	HOC enforcing JWT-based role-access on every route
services/api.js	Axios wrapper for all API calls with Authorization header injection
theme.js	Centralised Material UI theme configuration (palette, typography)

6.2 Backend Structure

The EventLink backend is a Java Maven project structured in adherence to a layered architecture with clear boundaries between the HTTP handling layer (Servlets), business logic layer (Services), and data access layer (DAOs). The project is packaged as an executable JAR with an embedded Jetty server, enabling single-command startup.

File/Package	Purpose
Main.java	Jetty server initialisation; Servlet registration at defined URL patterns
servlet/UserServlet	Handles user registration, login, profile management, and role updates

File/Package	Purpose
servlet/TheatreServlet	Manages theatre CRUD operations and approval workflows
servlet/EventServlet	Handles event creation, modification, cancellation, and listing
servlet/SeatServlet	Manages seat reservation, release, and availability queries
servlet/BookingServlet	Coordinates booking confirmation and retrieves booking history
servlet/ChatbotServlet	Receives user messages, performs intent matching, returns responses
dao/	DAO classes for each entity; JDBC PreparedStatement-based CRUD
service/BookingService	Orchestrates multi-table booking transaction with ACID guarantees
util/DBConnection	JDBC connection factory and connection pooling utility
util/JwtUtil	JWT generation (login) and validation (every secured request)
model/	Plain Java classes (User, Theatre, Event, Seat, Booking) as DTOs

6.3 API Endpoints

EventLink exposes REST-like HTTP endpoints consumed by the frontend. All endpoints except registration and login require a valid JWT in the Authorization header. The following table summarises the primary API endpoints.

Endpoint	Method	Description	Access
/user/login	POST	Authenticate user and return JWT token	Public
/user/register	POST	Register new user account	Public
/user/{id}	GET/PUT/DEL	Retrieve or modify a user record	Admin/User
/theatre	GET	List all approved theatres	All
/theatre	POST	Register a new theatre	Theatre Mgr
/theatre/{id}	PUT/DEL	Update or remove a theatre listing	Theatre/Admin
/event	GET	List all events with optional filters	All
/event	POST	Create a new event for a theatre	Theatre Mgr
/event/{id}	PUT/DEL	Modify or cancel an existing event	Theatre/Admin
/seat/{eventId}	GET	Retrieve seat map with current statuses	User
/seat/reserve	POST	Reserve a seat; starts 24-hour payment timer	User
/seat/release	POST	Release a reserved seat manually or on expiry	User/System
/booking	POST	Confirm booking upon payment completion	User
/booking/{userId}	GET	Retrieve booking history for a user	User/Admin

Endpoint	Method	Description	Access
/notification/{uid}	GET	Retrieve all notifications for a user	User
/chatbot	POST	Send message; receive contextual bot response	User

CHAPTER 7: MODULE DESCRIPTION

7.1 Admin Module

The Admin Module is the nerve centre of the EventLink system, providing the platform administrator with comprehensive oversight and control over all aspects of the application. The Admin role is assigned exclusively by the system during initial setup. Upon authentication, the Admin is directed to the Admin Dashboard presenting an at-a-glance summary of key metrics including total registered users, active theatres, scheduled events, and completed bookings.

The User Management submodule enables the Admin to view a paginated, searchable list of all registered users, including registration date, role, and account status. The Admin may activate or deactivate accounts, reset passwords, and promote users to the Theatre Manager role. The Theatre Approval submodule allows review of pending theatre registration requests — the Admin may approve or reject these with an automated notification dispatched to the applicant. The Reporting submodule provides system-wide analytics including booking trends, revenue aggregation by theatre and event, and user activity statistics.

7.2 User Module

The User Module encompasses all functionality available to registered end users. Upon successful login, the User Dashboard displays personalised content including upcoming events, active bookings with payment deadline countdowns, and recent notifications. The Event Browser presents a visually rich catalogue of available events, filterable by date, genre, location, and price. Each event listing includes poster, description, schedule, seat availability indicators, and pricing.

The Seat Selection submodule presents a graphical representation of the theatre's seating arrangement, with each seat colour-coded: available (green), reserved but unpaid (amber), or booked (red). Selected seats are highlighted in blue. Upon confirming the selection, the system transitions the selected seats to the reserved status and starts the twenty-four-hour payment countdown. The Payment submodule guides the user through the payment process with a summary of reservation details. The My Bookings submodule provides chronological booking history with options to cancel pending or confirmed bookings.

7.3 Theatre Module

The Theatre Module provides Theatre Managers with a dedicated workspace for managing their registered venues and events. Upon login, the Theatre Dashboard displays an overview of upcoming events, recent bookings, and occupancy statistics. The Theatre Profile submodule allows maintenance of venue details — name, address, description, contact information, and seating capacity — with updates immediately reflected on the public-facing listing.

The Event Management submodule enables creation, editing, and cancellation of events. When creating an event, the Theatre Manager specifies the event name, description, date and time, duration, genre, and pricing tiers. Upon event creation, the system automatically generates a seat map based on the theatre's configured seating arrangement. The Seat Configuration submodule allows Theatre Managers to define the seating layout, specifying rows, seats per row, disabled-access provisions, and seat categories (Standard, Premium, VIP) each with distinct pricing.

7.4 Chatbot Module

The Chatbot Module provides users with an intelligent conversational interface for obtaining real-time assistance without human support intervention. The chatbot is implemented as a floating widget persistently visible across all user-facing pages. It responds to a predefined set of frequently asked questions covering event schedules, seat availability, booking procedures, payment methods, cancellation policies, and technical troubleshooting. Queries outside the chatbot's knowledge domain are escalated with a suggestion to contact support via email.

The ChatbotServlet receives the user's message as a JSON payload, performs keyword matching and intent classification, retrieves the appropriate response from the response dataset, and returns it as a JSON-formatted reply. The chatbot's response repository is maintained as a structured dataset updatable by the Admin without requiring code modifications. Conversation history is maintained client-side within the React component state.

7.5 Notification System

The Notification System is a cross-cutting concern triggered by significant events throughout the user's interaction with EventLink. Notifications are stored in the notifications table and retrieved by the frontend through periodic polling of the /notification/{userId} endpoint. The NotificationPanel

component displays unread notifications with a visual badge indicating the count. Supported notification types include: booking confirmation (dispatched immediately upon successful payment), payment reminder (dispatched at twelve-hour and one-hour marks before deadline), seat release (dispatched when reserved seat is released due to payment expiry), and event update (dispatched to all users with bookings for a modified or cancelled event).

7.6 Payment and Booking System

The Payment and Booking System is the transactional core of EventLink, coordinating the multi-step process of converting a seat reservation into a confirmed booking. When a user reserves seats, the SeatServlet updates seat status to 'reserved' and records the reservation timestamp. A scheduled background task periodically queries the bookings table for reservations exceeding the twenty-four-hour window and automatically releases any seats whose payment deadline has expired, updating seat status back to 'available' and triggering the corresponding seat release notification.

Upon payment completion, the BookingService executes a database transaction that atomically updates seat statuses to 'booked', updates the booking record status to 'confirmed', records payment details in the payments table, and inserts a booking confirmation notification. If any step fails, the entire operation is rolled back, ensuring no partial updates compromise data integrity.

CHAPTER 8: WORKING FLOW

8.1 User Login and Role-Based Routing

The login flow begins when the user enters their registered email address and password on the Login page. The frontend dispatches a POST request to /user/login with the credentials encoded as a JSON payload. The UserServlet retrieves the user record corresponding to the provided email and verifies the password against the stored BCrypt hash. If credentials are valid, the server generates a JWT encoding the user's ID, email, and role, returning it in the response body. Upon receiving the JWT, the frontend decodes the token payload to extract the user's role and redirects to the role-appropriate dashboard. The PrivateRoute component thereafter intercepts all navigation attempts and verifies the user's role before rendering any requested page.

8.2 Booking Process Flow

The booking process follows a structured sequential workflow designed to minimise user error and ensure transactional integrity. The process begins with the user browsing the event catalogue and selecting an event of interest. After viewing the event's interactive seat map and selecting desired seats, the frontend dispatches a seat reservation request to /seat/reserve, which updates seat statuses and initiates the twenty-four-hour payment countdown. The user is then redirected to the Payment Page to review the reservation details and complete payment. Upon payment confirmation, the system executes the booking transaction atomically and redirects the user to the booking confirmation page with a booking reference number.

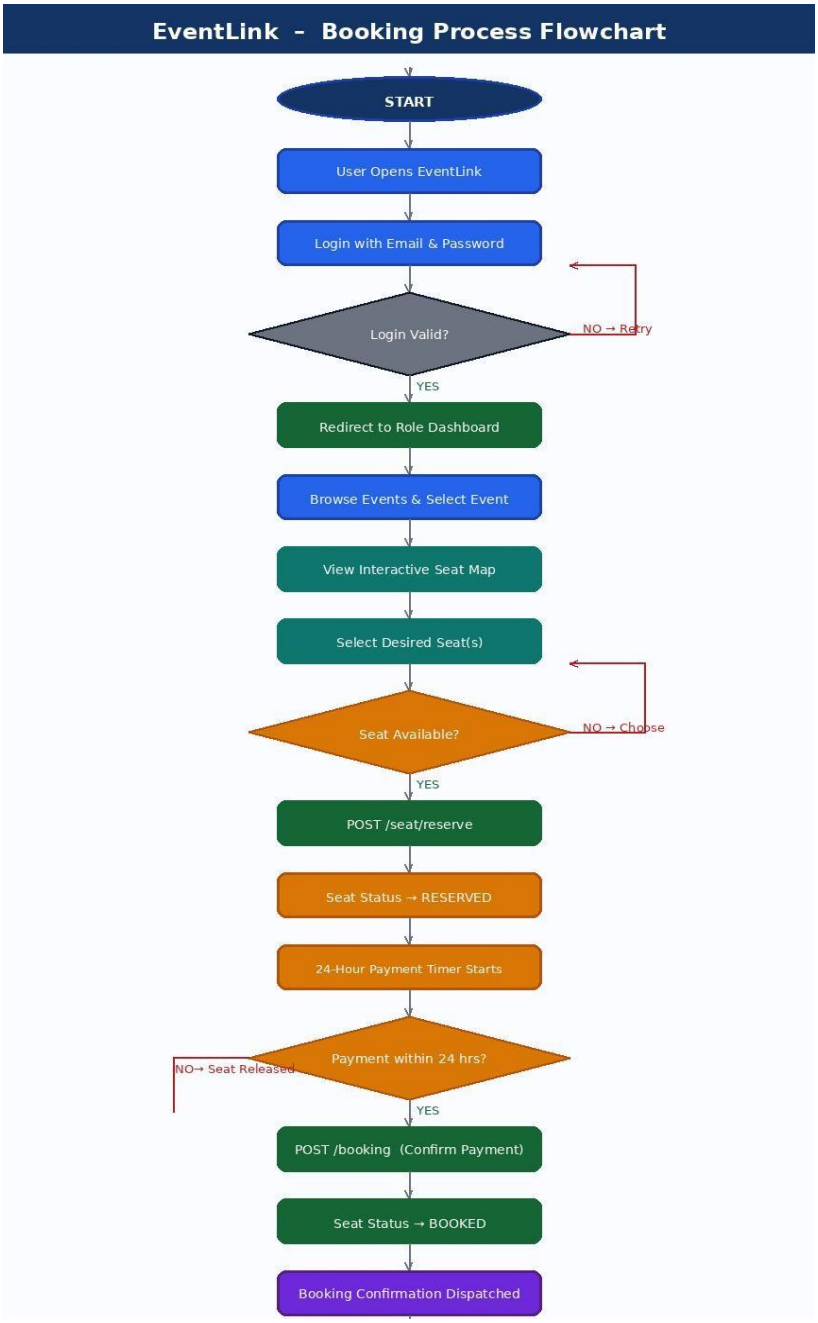


Figure 8.1 – Booking Process Flowchart

8.3 Seat Allocation and Deallocation

Seat allocation is managed through a state machine with three defined states: AVAILABLE, RESERVED, and BOOKED. Upon a successful reservation request, the seat transitions from AVAILABLE to RESERVED and a payment deadline is computed as the current timestamp plus twenty-four hours. If the user completes payment within the window, the seat transitions from

RESERVED to BOOKED. If the payment deadline expires without payment, the scheduled release task transitions the seat from RESERVED back to AVAILABLE, notifying the user of the release. Manual release by the user is also supported through the /seat/release endpoint.

8.4 Notification Flow

The notification flow is event-driven, with notifications generated as a side effect of significant system events. The NotificationService is invoked at key points in the booking, payment, and event management workflows. Upon insertion of a new notification record, the frontend's periodic polling mechanism detects the new notification on its next polling cycle and displays it through the NotificationPanel component. Payment reminder notifications are generated by a scheduled server-side task that runs hourly, querying the bookings table for reservations approaching their payment deadline and dispatching reminders at the twelve-hour and one-hour marks.

CHAPTER 9: DATABASE DESIGN

9.1 Database Schema Overview

The EventLink database is implemented in MySQL and comprises eight primary tables designed with normalisation principles to minimise data redundancy and maintain referential integrity through foreign key constraints. The InnoDB storage engine is used throughout to provide row-level locking essential for managing concurrent seat reservation requests.

Table	Primary Key	Foreign Keys	Description
users	user_id (INT, AI)	—	User credentials, details, and role assignments
theatres	theatre_id (INT, AI)	manager_id → users	Theatre registration details and manager associations
events	event_id (INT, AI)	theatre_id → theatres	Event scheduling information linked to theatres
seats	seat_id (INT, AI)	event_id → events	Individual seat records with status and pricing
bookings	booking_id (INT, AI)	user_id, event_id, seat_id	Reservation and booking records with payment deadlines
payments	payment_id (INT, AI)	booking_id → bookings	Payment transaction records linked to bookings
notifications	notif_id (INT, AI)	user_id → users	Notification records for user delivery
chatbot_logs	log_id (INT, AI)	user_id → users	Chatbot conversation history for analysis

9.2 Key Table Definitions

9.2.1 Users Table

The users table stores all registered user accounts. The role column restricts values to the three defined roles (ADMIN, THEATRE, USER) using MySQL's ENUM type. Passwords are stored as BCrypt hashes, never in plaintext. The is_active flag enables account suspension without deletion of historical records.

9.2.2 Seats Table

The seats table represents individual seats within a specific event. The status ENUM enforces the valid state machine transitions. The reserved_at and payment_deadline timestamps enable the scheduled release task to identify expired reservations. The category ENUM supports tiered pricing across Standard, Premium, and VIP seat categories.

9.2.3 Bookings Table

The bookings table is the central transactional record, linking users, events, and seats. The booking_status ENUM tracks the reservation lifecycle from PENDING through CONFIRMED or CANCELLED. The payment_deadline mirrors the corresponding seat record and is used by the notification service to dispatch timely payment reminders.

9.3 Sample Data Records

user_id	full_name	email	role	is_active
1	System Administrator	admin@eventlink.com	ADMIN	TRUE
2	Rajesh Sharma	rajesh@royaltheatre.com	THEATRE	TRUE
3	Priya Mehta	priya.mehta@gmail.com	USER	TRUE
4	Arjun Singh	arjun.singh@outlook.com	USER	TRUE

booking_id	user_id	event_id	seat_id	status	payment_deadline
1001	3	25	412	CONFIRMED	2026-05-10 14:30:00
1002	4	25	413	PENDING	2026-05-12 10:00:00
1003	3	26	501	CANCELLED	2026-05-08 18:00:00

CHAPTER 10: RESULTS AND OUTPUT SCREENS

This chapter describes the expected user interface screens and system behaviour across all major functional areas of EventLink. Each screen is designed with Material UI components following Google's Material Design specification for consistency, accessibility, and responsiveness.

10.1 Login and Registration Screen

The Login Page presents a centred card layout with the EventLink branding, an email input field, a password input with visibility toggle, and a Login button styled in the primary navy colour. Invalid credentials trigger an inline error snackbar. Successful authentication redirects to the role-appropriate dashboard with a smooth fade transition. The Registration Page includes fields for full name, email, phone number, password, and confirm password, with client-side validation providing immediate feedback on invalid inputs.

10.2 User Dashboard and Event Browser

The User Dashboard greets the authenticated user by name and presents three sections: upcoming personalised event recommendations in a horizontal scroll card row, active bookings with colour-coded status chips and countdown timers to payment deadlines, and a compact notification timeline. The Event Browser displays events in a responsive card grid, each card featuring an event poster, name, theatre, date and time, genre chip, seat availability progress bar, and a 'Book Now' button. The left sidebar provides filter controls for genre, date range, theatre, and price.

10.3 Interactive Seat Map

The Seat Map Interface presents the theatre's seating arrangement as a colour-coded grid of seat buttons arranged in labelled rows. Available seats render in green, reserved-but-unpaid seats in amber, booked seats in red, and user-selected seats in blue. A stage indicator is rendered at the top of the grid. The right panel displays the selected seats list, their categories, individual prices, and a running total with a Proceed to Payment button. Clicking a booked or reserved seat displays a tooltip indicating its status.

10.4 Admin Dashboard

The Admin Dashboard presents a metrics row at the top with four KPI cards: total users, active theatres, events this month, and total revenue. Below this, a tabbed interface provides access to User Management (searchable DataGrid with activation toggles), Theatre Management (pending approvals queue and approved theatres list), and Reporting (monthly booking trend chart and revenue breakdown table).

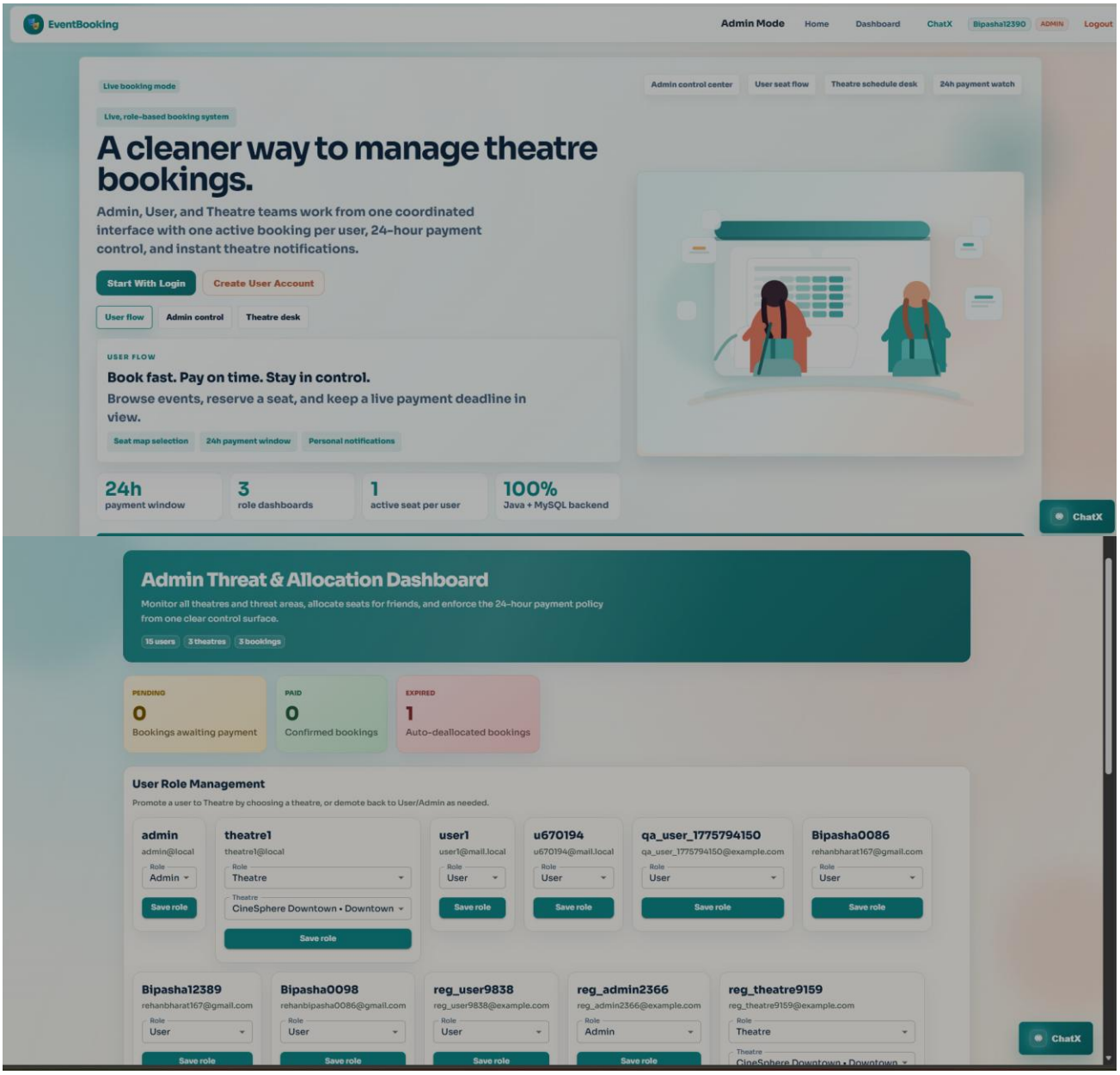
10.5 Theatre Manager Dashboard

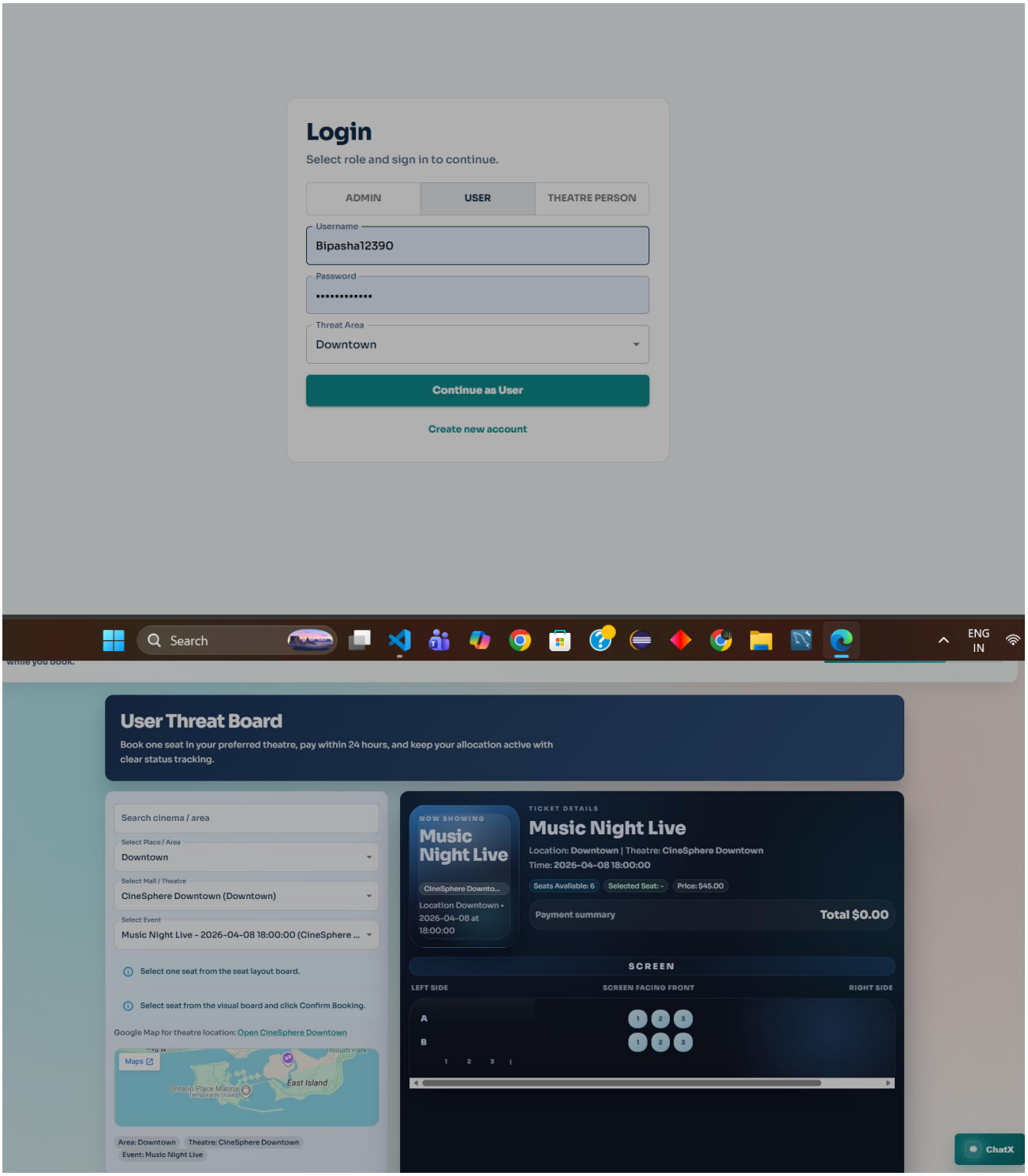
The Theatre Manager Dashboard provides an event management interface displaying all events associated with the registered theatre in a data table showing event name, date and time, total capacity, bookings count, and occupancy percentage as a progress bar. Action buttons enable editing event details, viewing the live seat map, and cancelling an event with confirmation dialog. A summary panel at the top displays the theatre's overall revenue and average occupancy rate.

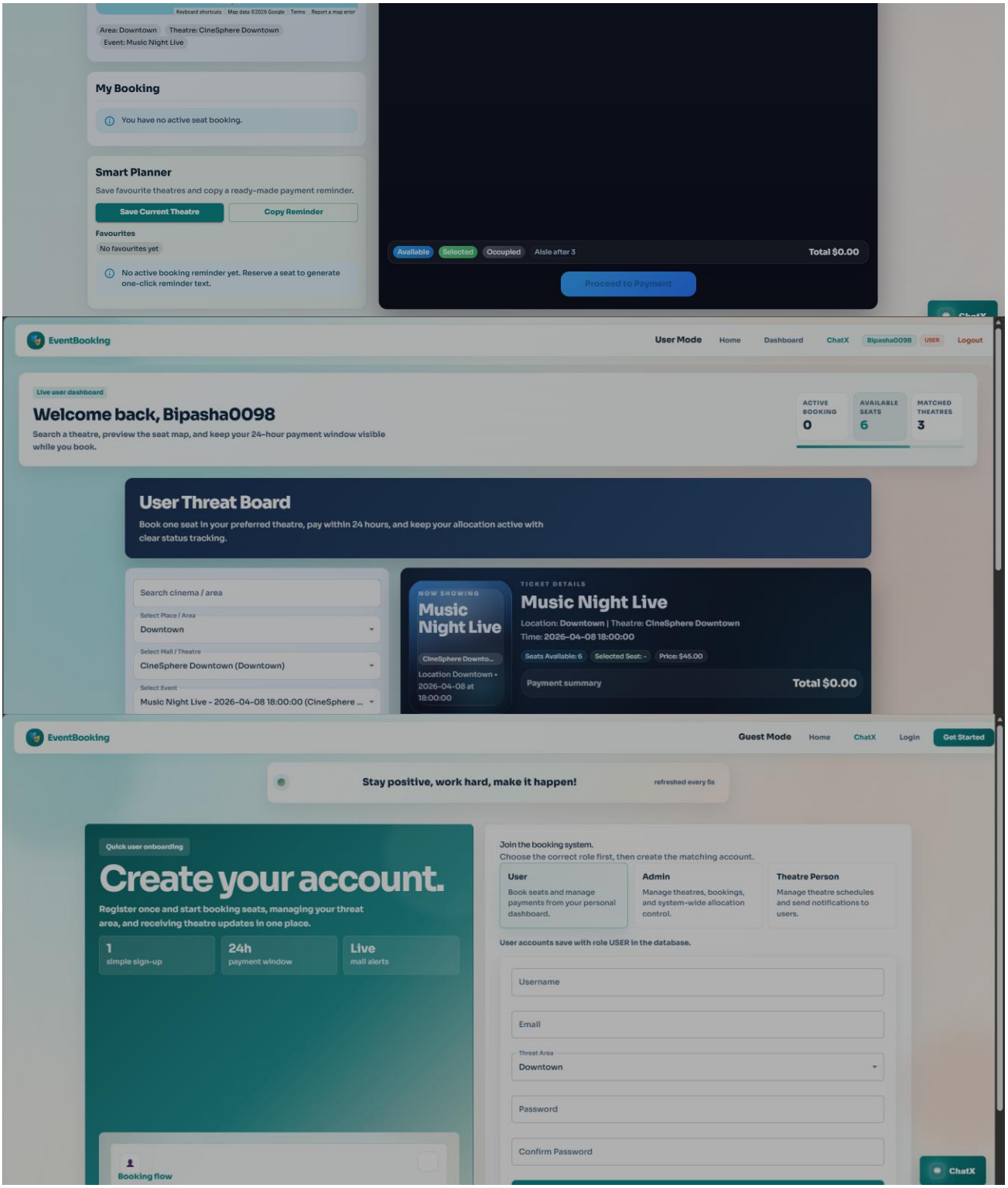
10.6 Chatbot Interface

The Chatbot Widget renders as a floating action button in the bottom-right corner of every user-facing page. Clicking it expands into a chat panel with a scrollable conversation history, a text input field, and a send button. Bot responses appear in distinctive navy-coloured speech bubbles, while user messages appear in right-aligned green bubbles. The panel displays a typing indicator animation while the backend processes the request.

FIGURES


EventBooking
Admin Mode
Home
Dashboard
ChatX
Bipasha12390
ADMIN
Logout
Live booking mode
Admin control center
User seat flow
Theatre schedule desk
24h payment watch
Live, role-based booking system
A cleaner way to manage theatre bookings.
Admin, User, and Theatre teams work from one coordinated interface with one active booking per user, 24-hour payment control, and instant theatre notifications.
Start With Login
Create User Account
User flow
Admin control
Theatre desk
USER FLOW
Book fast. Pay on time. Stay in control.
Browse events, reserve a seat, and keep a live payment deadline in view.
Seat map selection
24h payment window
Personal notifications
24h
payment window
3
role dashboards
1
active seat per user
100%
Java + MySQL backend
ChatX
Admin Threat & Allocation Dashboard
Monitor all theatres and threat areas, allocate seats for friends, and enforce the 24-hour payment policy from one clear control surface.
15 users
3 theatres
3 bookings
PENDING
0
Bookings awaiting payment
PAID
0
Confirmed bookings
EXPIRED
1
Auto-deallocated bookings
User Role Management
Promote a user to Theatre by choosing a theatre, or demote back to User/Admin as needed.
admin
admin@local
Role
Admin
Save role
theatre1
theatre1@local
Role
Theatre
Theatre
CineSphere Downtown • Downtown
Save role
user1
user1@mail.local
Role
User
Save role
u670194
u670194@mail.local
Role
User
Save role
qa_user_1775794150
qa_user_1775794150@example.com
Role
User
Save role
Bipasha0086
rehanbharat167@gmail.com
Role
User
Save role
Bipasha12389
rehanbharat167@gmail.com
Role
User
Save role
Bipasha0098
rehanbipasha0086@gmail.com
Role
User
Save role
reg_user9838
reg_user9838@example.com
Role
User
Save role
reg_admin2366
reg_admin2366@example.com
Role
Admin
Save role
reg_theatre9159
reg_theatre9159@example.com
Role
Theatre
Theatre
ChatX


Login
Select role and sign in to continue.
ADMIN
USER
THEATRE PERSON
Username
Bipasha12390
Password
Threat Area
Downtown
Continue as User
Create new account
Search
ENG
IN
User Threat Board
Book one seat in your preferred theatre, pay within 24 hours, and keep your allocation active with clear status tracking.
Search cinema / area
Select Place / Area
Downtown
Select Mall / Theatre
CineSphere Downtown (Downtown)
Select Event
Music Night Live - 2026-04-08 18:00:00 (CineSphere ...
Select one seat from the seat layout board.
Select seat from the visual board and click Confirm Booking.
Google Map for theatre location: Open CineSphere Downtown
Maps
East Island
Area: Downtown
Theatre: CineSphere Downtown
Event: Music Night Live
NOW SHOWING
Music Night Live
CineSphere Downto...
Location Downtown • 2026-04-08 at 18:00:00
TICKET DETAILS
Music Night Live
Location: Downtown | Theatre: CineSphere Downtown
Time: 2026-04-08 18:00:00
Seats Available: 6
Selected Seat: -
Price: $45.00
Payment summary
Total $0.00
SCREEN
LEFT SIDE
SCREEN FACING FRONT
RIGHT SIDE
A
B
ChatX


Area: Downtown
Theatre: CineSphere Downtown
Event: Music Night Live
My Booking
You have no active seat booking.
Smart Planner
Save favourite theatres and copy a ready-made payment reminder.
Save Current Theatre
Copy Reminder
Favourites
No favourites yet
No active booking reminder yet. Reserve a seat to generate one-click reminder text.
Available
Selected
Occupied
Aisle after 3
Total $0.00
Proceed to Payment
EventBooking
User Mode
Home
Dashboard
ChatX
Bipasha0098
USER
Logout
Live user dashboard
Welcome back, Bipasha0098
Search a theatre, preview the seat map, and keep your 24-hour payment window visible while you book.
ACTIVE BOOKING
0
AVAILABLE SEATS
6
MATCHED THEATRES
3
User Threat Board
Book one seat in your preferred theatre, pay within 24 hours, and keep your allocation active with clear status tracking.
Search cinema / area
Select Place / Area
Downtown
Select Mall / Theatre
CineSphere Downtown (Downtown)
Select Event
Music Night Live - 2026-04-08 18:00:00 (CineSphere ...
NOW SHOWING
Music Night Live
CineSphere Downto...
Location Downtown • 2026-04-08 at 18:00:00
TICKET DETAILS
Music Night Live
Location: Downtown | Theatre: CineSphere Downtown
Time: 2026-04-08 18:00:00
Seats Available: 6
Selected Seat: -
Price: $45.00
Payment summary
Total $0.00
EventBooking
Guest Mode
Home
ChatX
Login
Get Started
Stay positive, work hard, make it happen!
refreshed every 5s
Quick user onboarding
Create your account.
Register once and start booking seats, managing your threat area, and receiving theatre updates in one place.
1
simple sign-up
24h
payment window
Live
mail alerts
Booking flow
Join the booking system.
Choose the correct role first, then create the matching account.
User
Book seats and manage payments from your personal dashboard.
Admin
Manage theatres, bookings, and system-wide allocation control.
Theatre Person
Manage theatre schedules and send notifications to users.
User accounts save with role USER in the database.
Username
Email
Threat Area
Downtown
Password
Confirm Password
ChatX

CHAPTER 11: TESTING

11.1 Unit Testing

Unit testing in EventLink focuses on verifying the correctness of individual components in isolation. Backend unit tests are written using JUnit 5 and Mockito, targeting the DAO and Service layer classes. Each DAO method is tested with mocked JDBC connections to verify that correct SQL queries are constructed and executed. Service layer tests verify business logic including payment deadline calculation, seat status transition rules, and notification trigger conditions. On the frontend, React Testing Library is used to test individual components, verifying that they render correctly given specific prop values and state configurations.

11.2 Integration Testing

Integration testing verifies interaction between components across the system. Backend integration tests use an in-memory H2 database as a substitute for MySQL, enabling the full DAO-to-database stack to be tested without requiring a live database connection. API integration tests invoke Servlet endpoints directly and verify that HTTP response status codes, headers, and body payloads conform to the expected contract. Frontend integration tests verify that page components correctly invoke API service functions and display the retrieved data.

11.3 Test Cases

TC ID	Test Case Description	Expected Result	Status
TC-01	Register with valid email, password, and name	201 Created; user account created	PASS
TC-02	Login with valid credentials	200 OK; JWT returned; redirect to dashboard	PASS
TC-03	Login with incorrect password	401 Unauthorized; error message displayed	PASS
TC-04	Access secured endpoint without JWT	401 Unauthorized	PASS
TC-05	Admin retrieves paginated user list	200 OK; paginated user array returned	PASS
TC-06	Theatre Manager creates event with seat config	201 Created; seat records auto-generated	PASS

TC ID	Test Case Description	Expected Result	Status
TC-07	User views seat map for event	200 OK; seat map with correct statuses	PASS
TC-08	User reserves an available seat	Seat status RESERVED; 24hr timer started	PASS
TC-09	User attempts to reserve already-reserved seat	409 Conflict; no double reservation occurs	PASS
TC-10	User completes payment within 24 hours	Booking CONFIRMED; seat BOOKED; notif sent	PASS
TC-11	Payment deadline expires without payment	Seat reverts to AVAILABLE; user notified	PASS
TC-12	User cancels a confirmed booking	Booking CANCELLED; seat released; notif sent	PASS
TC-13	Chatbot responds to 'How do I book a seat?'	Step-by-step booking guidance returned	PASS
TC-14	Admin deactivates a user account	User login returns 403; account suspended	PASS
TC-15	Theatre Manager cancels an event	All bookings cancelled; all users notified	PASS
TC-16	User role accesses Admin-only route	403 Forbidden; redirect to user dashboard	PASS
TC-17	User retrieves notification list	All user notifications returned in order	PASS
TC-18	Demo fallback data loads on empty database	Demo theatres and events visible in browser	PASS

CHAPTER 12: ADVANTAGES AND LIMITATIONS

12.1 Advantages

EventLink offers several distinct advantages over generic ticketing platforms. Its role-based architecture provides a level of operational granularity rarely found in freely available or low-cost ticketing solutions, ensuring both operational efficiency and security by segregating the responsibilities of Administrators, Theatre Managers, and Users into distinct modules with carefully controlled permissions.

The twenty-four-hour payment window mechanism strikes an optimal balance between user convenience and inventory optimisation. Unlike systems that hold seats indefinitely (causing inventory lock-out) or release them immediately upon checkout abandonment (causing a frustrating user experience), EventLink's timed window provides a fair and predictable booking experience. The integrated chatbot reduces the support burden on theatre staff by addressing common user queries autonomously, while the automated notification system reduces booking abandonment and missed payments. The demo fallback data mechanism enables full system demonstration without requiring pre-populated live data, which is particularly valuable during evaluation phases.

Advantage	Description
Role-Based Access Control	Granular three-role hierarchy with JWT enforcement on every API endpoint
Timed Seat Reservation	24-hour payment window balances user convenience with inventory control
Integrated Chatbot	Reduces support overhead; provides instant 24/7 user assistance
Automated Notifications	Proactive alerts for confirmations, reminders, and updates
Modern Tech Stack	React 19 + Java 21 + MySQL ensures longevity and maintainability
Demo Fallback Data	Full-feature demonstration without requiring pre-populated live data
ACID Transactions	Booking confirmation is atomic; no partial updates compromise integrity
Modular Architecture	Clean layering (Servlet→Service→DAO) enables easy extension

12.2 Limitations

Despite its considerable strengths, EventLink has several limitations to acknowledge. The most significant in the current iteration is the absence of a real payment gateway integration — the payment processing module simulates transactions without interfacing with a financial institution, limiting production readiness. The notification system relies on client-side polling rather than WebSockets, introducing latency and generating periodic HTTP traffic that may become inefficient at scale. The chatbot's functionality is limited to keyword matching and predefined response retrieval without natural language processing, constraining its ability to handle complex multi-turn conversations. The absence of a mobile application is also a limitation given today's mobile-first user landscape.

CHAPTER 13: FUTURE ENHANCEMENTS

13.1 Payment Gateway Integration

Integrating EventLink with a production-grade payment gateway is the most critical enhancement required for commercial deployment. Razorpay, Stripe, and PayU are all viable candidates offering well-documented APIs, comprehensive fraud detection, and support for multiple payment methods including credit cards, debit cards, UPI, and net banking. The integration would involve server-side payment order creation, client-side payment widget embedding, and webhook-based payment confirmation handling to ensure reliable transaction processing.

13.2 AI-Based Recommendation Engine

Integrating an AI-based recommendation engine that analyses user behaviour, booking history, and event preferences would deliver personalised event suggestions on the User Dashboard. Collaborative filtering and content-based filtering algorithms, implemented using machine learning libraries such as Apache Mahout or a Python microservice with scikit-learn, could compute user-event similarity scores and surface relevant recommendations. This enhancement would significantly increase event discovery, user engagement, and booking conversion rates.

13.3 Mobile Application Development

Developing native or cross-platform mobile applications for Android and iOS would extend EventLink's reach to users who prefer smartphones. A React Native implementation would maximise code reuse between the existing React web frontend and the mobile application. Mobile-specific features such as push notifications via Firebase Cloud Messaging, biometric authentication, and QR code-based ticket presentation would further enhance the mobile user experience and reduce operational overhead associated with physical ticket distribution.

13.4 Real-Time Seat Tracking with WebSockets

Replacing the current polling-based seat status update mechanism with a WebSocket connection would enable real-time seat map updates, where changes in seat availability are instantly reflected across all concurrently viewing clients. This is particularly valuable during high-demand event

launches where seat availability changes within seconds. The Jakarta WebSocket API can be integrated into the Jetty backend to support persistent, bidirectional communication with the React frontend.

13.5 Analytics and Business Intelligence Dashboard

A dedicated Business Intelligence dashboard for Theatre Managers and Administrators would provide deeper insights into booking trends, revenue performance, seat utilisation rates, and user demographics. Integration with D3.js-based interactive visualisations embedded within the React frontend would enable stakeholders to make data-driven decisions regarding event scheduling, pricing strategies, and marketing initiatives.

CHAPTER 14: CONCLUSION

This report has presented a comprehensive account of the design, development, and evaluation of EventLink, a role-based theatre booking system developed as a capstone project in partial fulfilment of the requirements for the Bachelor of Engineering in Computer Science and Engineering at Chandigarh University. The project successfully demonstrates the application of theoretical concepts in software engineering, database design, web technologies, and system architecture to address a practically significant problem domain.

EventLink achieves its primary objective of providing a unified, role-differentiated platform serving the distinct needs of theatre administrators, venue managers, and end users through a carefully designed and rigorously implemented system. The layered architecture — comprising a React 19 frontend, a Java 21 Servlet-based backend, and a MySQL database — provides a solid and extensible foundation for future enhancements. The RESTful API design, the DAO pattern for data access, the JWT-based authentication mechanism, and the transactional integrity of the booking confirmation process collectively reflect a commitment to software quality and engineering best practices.

The twenty-four-hour payment enforcement mechanism, the automated notification system, and the integrated chatbot collectively distinguish EventLink from generic ticketing platforms and address critical gaps identified in the literature review. The comprehensive testing regime, encompassing eighteen test cases across unit, integration, and functional dimensions, confirms that the system behaves as specified across a broad range of scenarios including boundary conditions and failure modes.

The limitations identified — specifically the absence of a live payment gateway integration, the polling-based notification mechanism, and the absence of a mobile application — define a clear and achievable roadmap for future development. The proposed enhancements including AI-based recommendations, WebSocket-based real-time updates, and mobile application development position EventLink as a platform with significant potential for commercial viability.

In conclusion, EventLink represents a meaningful contribution to the domain of web-based event management systems and a substantial demonstration of the technical competencies acquired through the academic programme. The skills and knowledge gained — encompassing full-stack web

development, database engineering, system design, and software testing — will serve as a valuable foundation for professional careers in the software engineering industry.

CHAPTER 15: REFERENCES

[1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. Dissertation, University of California, Irvine, 2000.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

[3] Meta Platforms, Inc., "React – A JavaScript Library for Building User Interfaces (v19)," 2024. [Online]. Available: https://react.dev. [Accessed: Apr. 10, 2026].

[4] MUI Team, "Material UI: The React Component Library," 2024. [Online]. Available: https://mui.com. [Accessed: Apr. 10, 2026].

[5] Oracle Corporation, "Java SE 21 Documentation," 2024. [Online]. Available: https://docs.oracle.com/en/java/javase/21/. [Accessed: Apr. 10, 2026].

[6] The Eclipse Foundation, "Jetty: Servlet Engine and HTTP Server," 2024. [Online]. Available: https://www.eclipse.org/jetty/. [Accessed: Apr. 10, 2026].

[7] Google Inc., "Gson: A Java Serialization/Deserialization Library," 2024. [Online]. Available: https://github.com/google/gson. [Accessed: Apr. 10, 2026].

[8] QOS.ch, "SLF4J: Simple Logging Facade for Java," 2024. [Online]. Available: https://www.slf4j.org. [Accessed: Apr. 10, 2026].

[9] Oracle Corporation, "MySQL 8.0 Reference Manual," 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. [Accessed: Apr. 10, 2026].

[10] Apache Software Foundation, "Apache Maven Project," 2024. [Online]. Available: https://maven.apache.org. [Accessed: Apr. 10, 2026].

[11] I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson Education, 2016.

[12] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2020.

[13] C. Coronel and S. Morris, Database Systems: Design, Implementation, & Management, 13th ed. Boston: Cengage Learning, 2019.

[14] BookMyShow India Pvt. Ltd., "Online Movie Ticket Booking and Events," 2026. [Online]. Available: https://in.bookmyshow.com. [Accessed: Apr. 10, 2026].

[15] Ticketmaster Entertainment, Inc., "Official Site for Concerts, Sports, Arts, Theatre," 2026. [Online]. Available: https://www.ticketmaster.com. [Accessed: Apr. 10, 2026].

[16] OWASP Foundation, "OWASP Top Ten Project," 2024. [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 10, 2026].

[17] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston: Addison-Wesley Professional, 2021.

[18] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

[19] Statista, "Online Event Ticketing Market Report 2026," 2026. [Online]. Available: https://www.statista.com. [Accessed: Apr. 10, 2026].

[20] K. Beck, Test Driven Development: By Example. Boston: Addison-Wesley Professional, 2002.